pynxtools-spm general table:

Parser	nxformatters	Input format	Documentation	Standard behaviour	Special behaviour	Formula / transformation	Notes
nanonins_dat (STS experiment)	nanonis_dat_sts.py	.dat	[No dedicated documentation link for each parser]. General reader/parser structure	Parses Nanonis .dat metadata and numerical data into structured key–value pairs, interprets units, reshapes data into columns, and maps the results into the target NXdata structure.	Uses header markers such as "[DATA]" and last_line to identify the data block; applies current sign correction in the formatter using an ELN-configured flip_sign value; calculates dI/dV through an external helper function.	Bias step size: ΔV = (V_end − V_start) / N_points (nanonis_dat_sts.py, lines 127–129). Current sign correction: I_corrected = flip_number × I (line 332). Differential conductance: dI/dV is calculated numerically using a second-order-accurate finite-difference method, implemented as Diff (axis=0, grid=V, acc=2) (nanonis_dat_sts.py, line 210; helpers.py, lines 489–492). The actual voltage values are used as the differentiation grid.	Check whether the data accumulator is reset between blocks (nanonis_dat parser, lines 211, 251, 255–257, 267–269); whether malformed rows or header/data mismatches cause data loss or errors; whether missing units are handled correctly (lines 103–104, 202–203); whether flip_number = 0 is unintentionally replaced by the default value (formatter, line 323); and whether the bias-step denominator should be \(N\) or \(N-1\)
nanonis_sxm (STM and AFM experiments)	nanonis_sxm_afm.py nanonis_sxm_stm.py	.sxm		Reads Nanonis .sxm files via nanonispy, extracts metadata and signals, flattens metadata into slash-separated paths, extracts unit labels, and maps channel information into a NeXus template. The STM formatter constructs scan geometry, X/Y pixel-centre axes, image/channel NXdata groups, and timestamps. The AFM formatter largely reuses the STM implementation. References: nanonis_sxm.py, lines 68–175; nanonis_sxm_stm.py, lines 75–465; nanonis_sxm_afm.py, lines 64–181.	Binary signal decoding is delegated to nanonispy. The parser extracts units from parentheses and expands the /DATA/INFO table into channel metadata. The STM formatter dynamically delegates bias-sweep construction to NanonisDatSTS and uses a module-level global (gbl_scan_ranges). AFM inherits and reuses most STM formatting logic. NXdata axes are ordered [Y, X], with Y index 0 and X index 1. References: nanonis_sxm.py, lines 41–175; nanonis_sxm_stm.py, lines 153–177 and 407–465; nanonis_sxm_afm.py, lines 77–181.	Scan step size: $\Delta x_i = R_i / N_i$, where R_i is the scan range and N_i is the number of points. Scan-region start/end positions are delegated to derive_scan_2d_start_end(), and pixel-centre coordinates to _pixel_centres(). Image data passes through to_intended_t(), which converts numeric strings and lists to numeric types/NumPy arrays where possible, then the formatter rearranges array axes for NXdata ([Y, X]). Calibration and offset are stored as attributes. Unit helpers validate units and shorten unit symbols; they do not demonstrate numerical conversion of signal values. The helper cal_dy_by_dx(y, x) computes a numerical derivative using findiff Diff(axis=0, grid=x, acc=2). End time: t_end = t_record + Δt_acquisition.	1. Investigate the range/point-count denominator (R/N versus R/(N−1)); the use of the X-axis unit for every step-size axis; possible state leakage through global gbl_scan_ranges; and runtime class monkey-patching for bias-sweep construction. 2. Check whether malformed /DATA/INFO rows, unexpected direction labels, duplicate keys, or metadata/signal path collisions cause silent data loss. 3. Verify whether calibration and offset should be applied to image values, inspect the inherited pixel-centre and scan-region formulas, and test timestamp parsing for missing or zero acquisition durations and malformed dates.
omicron_sm4 (STM experiment)	omicron_sm4_stm.py	.sm4		Reads the page metadata from the binary file with its own reader (RHK_* keys) and the images with gwyddionpy, then matches each image to its metadata by page ID. Writes a flat dict per page label (metadata, pixel-centre coordinates, image). OmicronSM4STM maps it to NXstm through a JSON config.	- Non-image pages (spectra): metadata only + warning, no data/coords - Image without matching channel: metadata only + warning - Keys ending in unit/units (case-insensitive) → /@unit - RHK_FileName replaced by the read path; RHK_PageID = first uint16 of GUID only - Strings not UTF-8 encodable (lone surrogate) → "" - PRM data: optional zlib, decoded cp437 - Offset = frame centre; RHK_Angle exported, not applied - Slow direction = sign(RHK_Yscale); row flip in OmicronBase (formatter, per docs)	Pixel-centre coordinates, per axis a = x, y: R_a = N_a · |s_a| a_i = o_a − R_a/2 + (i + ½) · |s_a|, i = 0 … N_a − 1 o = RHK_{X,Y}offset (frame centre), s = RHK_{X,Y}scale, N = RHK_{X,Y} size; coordinates always ascend. Z calibration: done by gwyddionpy (RHK_Zscale, RHK_Zoffset), not in the parser. Unit keys: …unit / …units → …/@unit Date: "MM/DD/YY" + time → "20YY-MM-DDThh:mm:ss.000"	1. (units|unit)$ over-matches, e.g. RHK_API_Units → RHK_API_/@unit 2. GUID byte order vs Gwyddion "Page ID"; mismatch silently drops the image 3. Duplicate Page IDs overwrite in the channel dict 4. Spectroscopy (RHK_PAGE_IV_*) is metadata only; intended for NXstm? 5. ValueError on truncated file is uncaught; _read_lockin swallows it 6. _read_low_pass_filter needs exactly 2 tokens 7. RHK_DateTime: hard-coded century, MM/DD/YY, no timezone 8. Confirm Z calibration is not applied twice (gwyddionpy + config) 9. float32 precision of offset/scale 10. Formatter (OmicronSM4STM/OmicronBase) not reviewed
bruker_flt (AFM experiment)	bruker_flt_afm.py	.flt		Reads the single channel with gwyddionpy and the INI-style header with its own reader, splits header values into value + unit and flattens everything to slash paths. BrukerFltAFM maps it to NXafm and derives the scan region, step sizes, fast-axis sign and X/Y axes.	- One file = one channel, one scan direction - Own header read keeps keys gwyddionpy drops (OffsetX/Y) and separates the two Leveling keys - PID gain "0.8 (1000.0)" keeps 0.8; loop constant dropped - Header values stay strings; units in UNIT_TO_SKIP → "" - OffsetX/Y read as frame centre (contradicts Gwyddion spmlabf.c) - Rotation exported, not applied; no slow direction in file → bare Y axis - Row flip + descending slow axis done in BrukerBase (docs)	Header value/unit: "v u" → v (string), unit u; PID gain "g (c)" → g (constant c dropped) Scan region (offset = frame centre): start/end = O ∓ R/2, for x and y Step size: Δ_a = R_a / N_a, a = x, y (skipped if N < 2) Axes: pixel centres, ascending, sized from the image shape Fast axis: ScanDirection forward → +x, backward → −x; slow axis unsigned Time: "Jun.17.2026 08:03:43" → ISO 8601 (no timezone)	1. Lines before first [section] lost silently 2. Duplicate keys in a section overwrite 3. DataOffset beyond first 8192 bytes not found 4. Empty "/@unit" = "" for skipped units 5. "2 nd order" parsed as value 2, unit "nd" 6. Slow direction is an assumption (docs: open question) 7. Rotation not applied 8. Check channel.data unit = si_unit_z for all channel types 9. Step size: R/N vs R/(N-1), compare with other parsers 10. Formatter (BrukerFltAFM/BrukerBase) not reviewed
bruker_spm (AFM experiment)	bruker_spm_afm.py	.spm		Reads the ASCII header into sections and the image layers with gwyddionpy, pairs layers with header entries in file order, scales each image to its channel unit and exposes every channel as forward and backward. BrukerSpmAFM builds the scan region, mesh scan and X/Y axes for NXafm.	- One file = all channels, both directions - One-direction channel exposed under both forward and backward - Rows left in gwyddionpy order; flip in BrukerBase - Keys: whitespace → "_"; "/" in key adds a path level - Y range = X range / Aspect Ratio (1:1 fallback); axes rebuilt from image shape if header disagrees (docs) - Offset = centre; rotation not applied	Z scaling: z_out = z_SI · (1 / m(u)) · k m(u) = base-unit magnitude of the sensitivity unit u (nm → 10⁻⁹); k = 1000 if the Z-scale unit is mV, else 1 Scan range: R_x = ScanSize, R_y = R_x / (a/b) for AspectRatio "a:b" Offsets (frame centre): start/end = O ∓ R/2 Step size: Δ_x = R_x / N_x, Δ_y = R_y / N_y Header: "V (c u/LSB) v u" → v, key/@units = u; "v u" → v, key/@unit = u	1. "@units" (scaled) vs "@unit" (others) 2. Regexes reject negative / exponent values (e.g. "-500 nm" stays a string) 3. Units with "/" (nm/V) not split 4. Positional image–channel pairing: shifts if gwyddionpy skips a layer 5. Duplicate channel+direction overwrite, Scan_list gets both entries 6. Fake backward data for one-direction channels 7. Channel title with "-" or "_" → layer dropped 8. Only "mV" prefix handled; pint failure → factor 1 silently; no signal unit exported 9. First matching sensitivity section wins 10. Other sections dropped; file read twice (memory) 11. Formatter (BrukerSpmAFM/BrukerBase) not reviewed
bruker_txt (AFM experiment)	bruker_txt_afm.py	.spm.txt		Reads the quoted ASCII export line by line: sections become metadata (value + unit), followed by the numeric columns as float arrays with units taken from the column names. BrukerTxtAFM derives approach/retrace range, point counts, step sizes, amplitude setpoint and timestamp for NXafm (point force scan).	- Extend/retract columns end in _Ex / _Rt; unit is part of the name - Metadata before first section has no section prefix - Column-count mismatch: error logged, loop stops, partial arrays kept - Any .txt input is routed here (docs) - Samps/line may hold 1 or 2 counts (approach first) (docs)	Unit from column name: Calc_Ramp_Ex_nm → nm (token after Ex/Rt); Time_s_Ex → s (token before) Range: s = r_e, e = r_(M−1), R = |e − s| (r = ramp array, M = length) Step size: Δ = R / (N − 1), for approach and retrace Amplitude setpoint: A_setpoint = A_ratio · A_ref Header: "V (c u/LSB) v u" → v, u; "v u" → v, u (leading minus allowed)	1. Mismatch → truncated arrays, only an error log 2. utf-8 only; µ/° in single-byte files raise UnicodeDecodeError 3. .split() on whitespace vs tab-separated columns 4. Unit from name: "Defl_Ex" → unit "Defl"; names containing Ex/Rt tokens; no unit validation 5. Unit key "/unit" vs "/@unit" in FLT/SPM 6. Scaled regex rejects negatives/exponents; "nm/V" and multi-word bracket values not split 7. Header detection needs ":"-free line directly after the end marker 8. R/(N-1) here vs R/N elsewhere 9. Check array length = Samps/line count 10. Amplitude setpoint if reference amplitude missing; any .txt accepted 11. Formatter (BrukerTxtAFM) not reviewed